<<route>>
PaymentRoutes
+POST /payments/connect/onboarding-link (AuthGuard)
+GET /payments/connect/status (AuthGuard)
+GET /payments/payment-methods (AuthGuard)
+POST /payments/customer-portal (AuthGuard)
+POST /payments/contracts/:id/platform-fee/intent (AuthGuard)
+POST /payments/contracts/:id/platform-fee/checkout-session (AuthGuard)
+POST /payments/contracts/:id/platform-fee/saved-card (AuthGuard)
+POST /payments/contracts/:cid/milestones/:mid/fund/intent (AuthGuard)
+POST /payments/contracts/:cid/milestones/:mid/fund/checkout-session (AuthGuard)
+POST /payments/contracts/:cid/milestones/:mid/fund/saved-card (AuthGuard)
+POST /payments/contracts/:cid/milestones/:mid/release (AuthGuard)
+POST /payments/contracts/:cid/milestones/:mid/refund (AuthGuard)
+POST /payments/disputes/:id/settle (AuthGuard)
+GET /payments/transactions (AuthGuard)
+POST /payments/checkout-sessions/:id/sync (AuthGuard)
+POST /payments/webhook (IsPublic)
maps to
calls
PaymentController
-paymentService: PaymentService
+createOnboardingLink(userId, body?)
+getConnectStatus(userId)
+getSavedPaymentMethods(userId)
+createCustomerPortalSession(userId, returnUrl?)
+createPlatformFeeIntent(userId, contractId)
+createPlatformFeeCheckoutSession(userId, contractId)
+chargeSavedCardForPlatformFee(userId, contractId, body?)
+createMilestoneFundIntent(userId, contractId, milestoneId)
+createMilestoneFundCheckoutSession(userId, contractId, milestoneId)
+chargeSavedCardForMilestoneFund(userId, contractId, milestoneId, body?)
+releaseMilestone(userId, contractId, milestoneId)
+refundMilestone(userId, contractId, milestoneId)
+settleDispute(userId, roleName, disputeId, body)
+getTransactions(userId, roleName, query)
+syncCheckoutSession(userId, sessionId)
+handleWebhook(req, signature)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
<<dto>>
CreatePaymentIntentResponseDTO
+clientSecret: string
+paymentIntentId: string
+amount: number
+currency: string
<<dto>>
CreateCheckoutSessionResponseDTO
+checkoutUrl: string
+sessionId: string
<<dto>>
ChargeSavedCardBodyDTO
+paymentMethodId?: string
<<dto>>
DisputeSettlementBodyDTO
+clientAmount: number
+freelancerAmount: number
+winningParty: 'CLIENT' | 'FREELANCER' | 'SPLIT'
+decisionReason: string
<<type>>
TransactionType
+id: number
+userId: number
+contractId: number | null
+milestoneId: number | null
+disputeId: number | null
+amount: number
+currency: string
+type: TransactionTypeEnum
+status: TransactionStatusEnum
+stripePaymentIntentId: string | null
+stripeTransferId: string | null
+createdAt: Date
PaymentService
-paymentRepo: PaymentRepository
-stripeService: StripeService
-logger: Logger
+getOnboardingLink(userId, returnUrl?)
+getConnectStatus(userId)
+getSavedPaymentMethods(userId)
+getCustomerPortalLink(userId, returnUrl?)
+createPlatformFeeIntent(userId, contractId)
+createPlatformFeeCheckoutSession(userId, contractId)
+chargeSavedCardForPlatformFee(userId, contractId, pmId?)
+createMilestoneFundIntent(userId, contractId, milestoneId)
+createMilestoneFundCheckoutSession(userId, contractId, milestoneId)
+chargeSavedCardForMilestoneFund(userId, contractId, milestoneId, pmId?)
+releaseMilestone(userId, contractId, milestoneId)
+refundMilestone(userId, contractId, milestoneId)
+settleDispute(callerUserId, callerRoleName, disputeId, body)
+getTransactions(userId, roleName, query)
+syncCheckoutSession(userId, sessionId)
+handleWebhook(payload, signature)
calls
calls
<<utility>>
StripeService
-stripe: Stripe
-logger: Logger
+createCustomer(email, name?)
+createExpressAccount(email, country?)
+createAccountLink(stripeAccountId, returnUrl, refreshUrl)
+createPaymentIntent({ amountInCents, currency, customerId, metadata })
+createTransfer({ amountInCents, destination, transferGroup, metadata })
+createRefund({ paymentIntentId, amountInCents, metadata })
+createCheckoutSession({ amountInCents, successUrl, cancelUrl, metadata })
+chargeOffSession({ amountInCents, customerId, paymentMethodId })
uses
PaymentRepository
-prisma: PrismaService
+findUserById(userId: number)
+updateUserStripe(userId: number, data)
+findContractById(contractId: number)
+updateContractPlatformFee(contractId, status, paidAt?)
+findMilestoneById(milestoneId: number)
+updateMilestonePaymentStatus(milestoneId, paymentStatus, status?)
+findDisputeById(disputeId: number)
+createTransaction(data)
+updateTransactionStatus(id, status, extra?)
+findTransactionByPaymentIntentId(paymentIntentId)
+getTransactions(query, filter)
maps to
maps to
maps to
<<schema>>
Transaction
+id: number [PK]
+userId: number [FK]
+contractId: number | null [FK]
+milestoneId: number | null [FK]
+disputeId: number | null [FK]
+amount: Decimal
+currency: string
+type: TransactionType
+status: TransactionStatus
+stripePaymentIntentId: string | null
+stripeTransferId: string | null
+stripeRefundId: string | null
+createdAt: Date
0..*
<<schema>>
Contract
+id: number [PK]
+clientId: number [FK]
+freelancerId: number [FK]
+platformFeeStatus: PlatformFeeStatus
+platformFeePaidAt: Date | null
+status: ContractStatus
1..*
<<schema>>
Milestone
+id: number [PK]
+contractId: number [FK]
+title: string
+amount: Decimal
+status: MilestoneStatus
+paymentStatus: MilestonePaymentStatus
1..*
<<schema>>
User
+id: number [PK]
+email: string
+stripeCustomerId: string | null
+stripeAccountId: string | null
+stripeOnboardingCompleted: boolean